



Modelado de Amenazas

Materias:

Ciberseguridad de Redes e Infraestructuras Críticas

Taller de Seguridad

Agenda

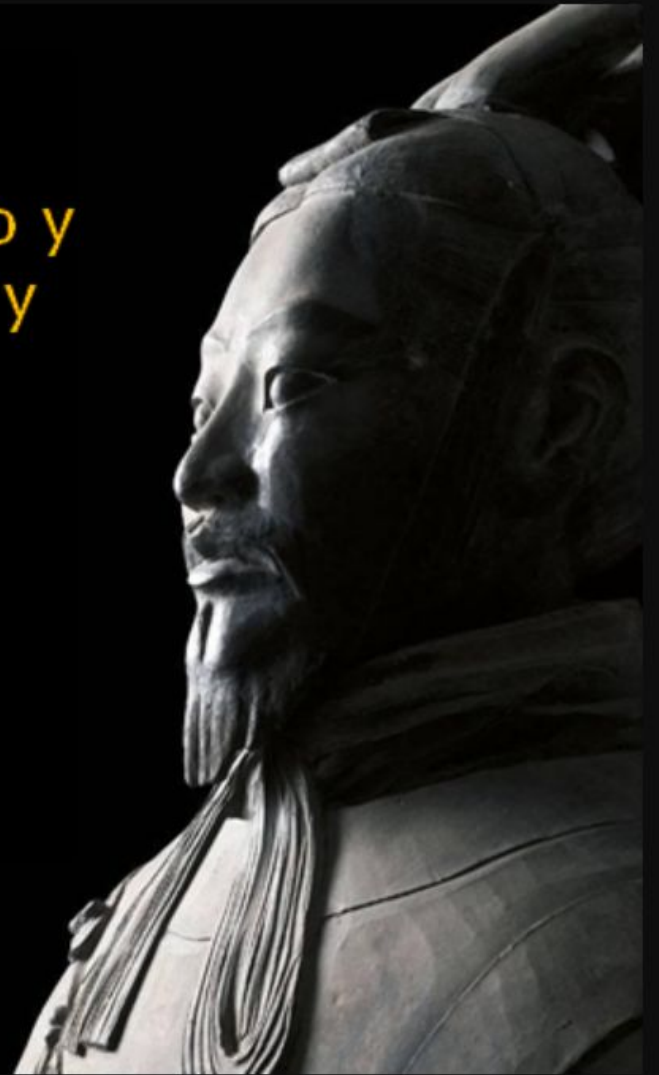
1. **Apertura y objetivos** ⇄
2. Motivación: por qué importa la seguridad
3. Propiedades / objetivos de protección
4. Marco conceptual de la Seguridad de la Información
5. Conceptos fundamentales para proteger activos
6. Mecanismos de seguridad
7. Seguridad desde el diseño
8. Integración sistémica de la seguridad
9. Cierre

Apertura y Objetivos

Respuesta a crisis de confianza y fraude sistémico

“Conoce a tu enemigo y
conócete a ti mismo, y
saldrás triunfador en
mil batallas”

— *Sun Tzu, "El Arte de la Guerra"*



La estrategia sin táctica es el camino más lento hacia la victoria. Las tácticas sin estrategia son el ruido antes de la derrota.

Sun Tzu

La estrategia es un patrón en una corriente de decisiones.

Henry Mintzberg

En la preparación para la batalla he encontrado que los planes son inútiles, pero la planificación es indispensable.

Dwight D. Eisenhower

- Aunque la incertidumbre puede volver obsoletos los planes detallados.
 - La preparación y la comprensión de objetivos y estrategias son fundamentales.
 - Proporcionan una base sólida para decidir en situaciones cambiantes y caóticas.
-

Pilares de la Seguridad de la Información

Confidencialidad

Integridad

Disponibilidad

No repudio

Propiedades auxiliares

Autenticidad

Autorización

Privacidad

Evidencia y control

Auditabilidad

Trazabilidad

Sostén ante fallas

Robustez

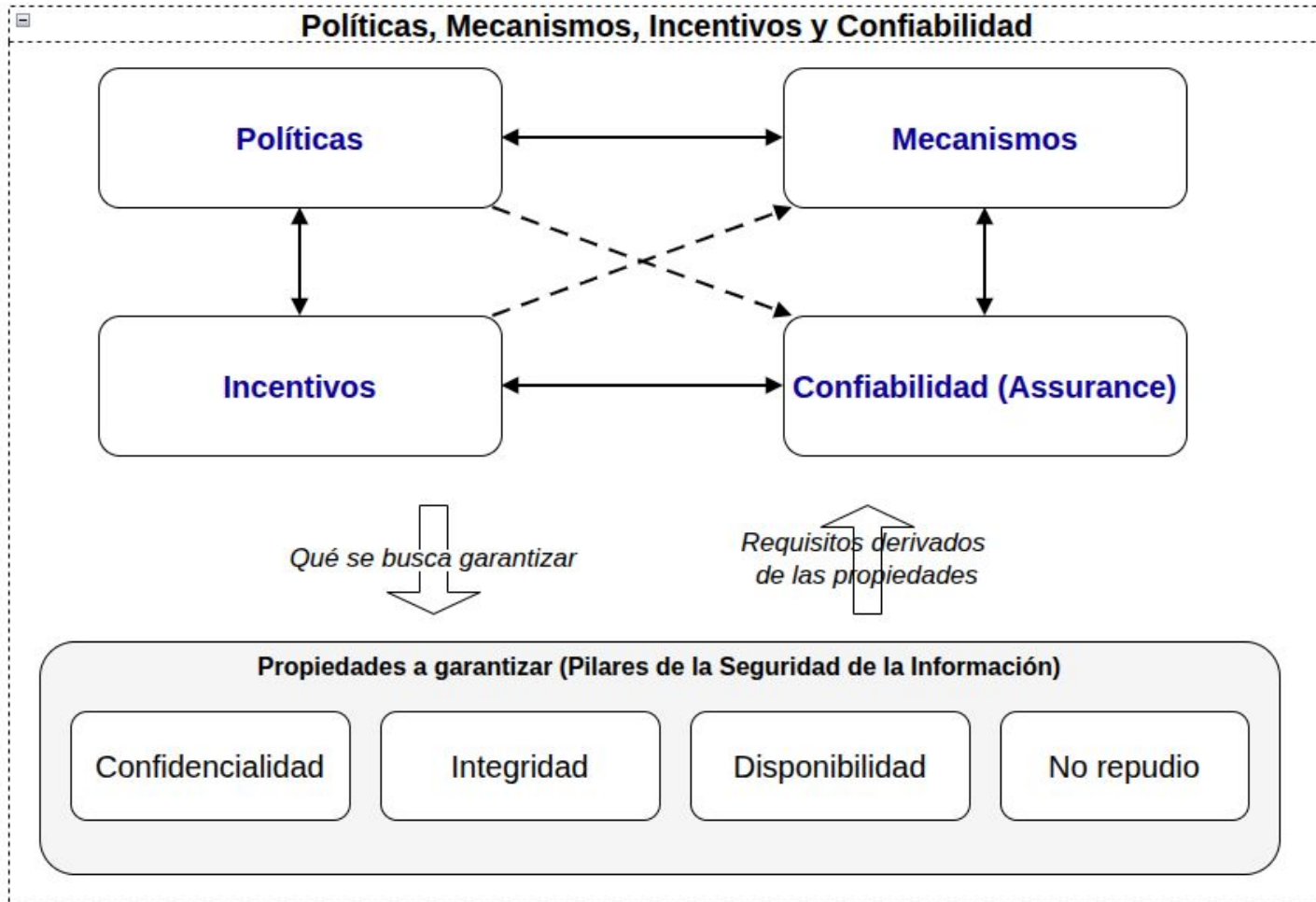
Resiliencia

Repaso

Pilares de la seguridad CIA



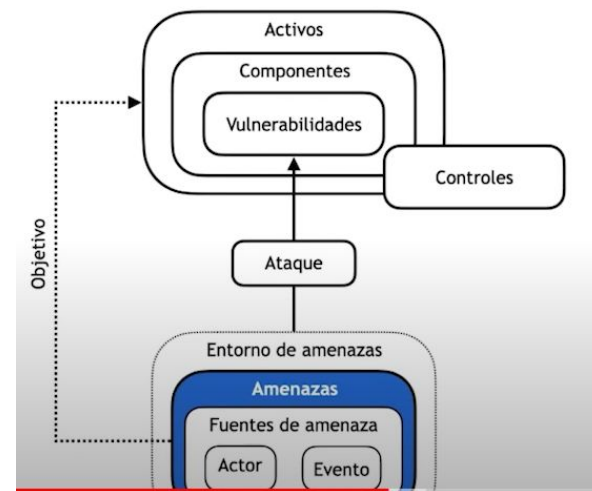
- Qué debe ser protegido?
- Por qué debe ser protegido?
- De qué debe ser protegido?
- Cómo protegerlo?



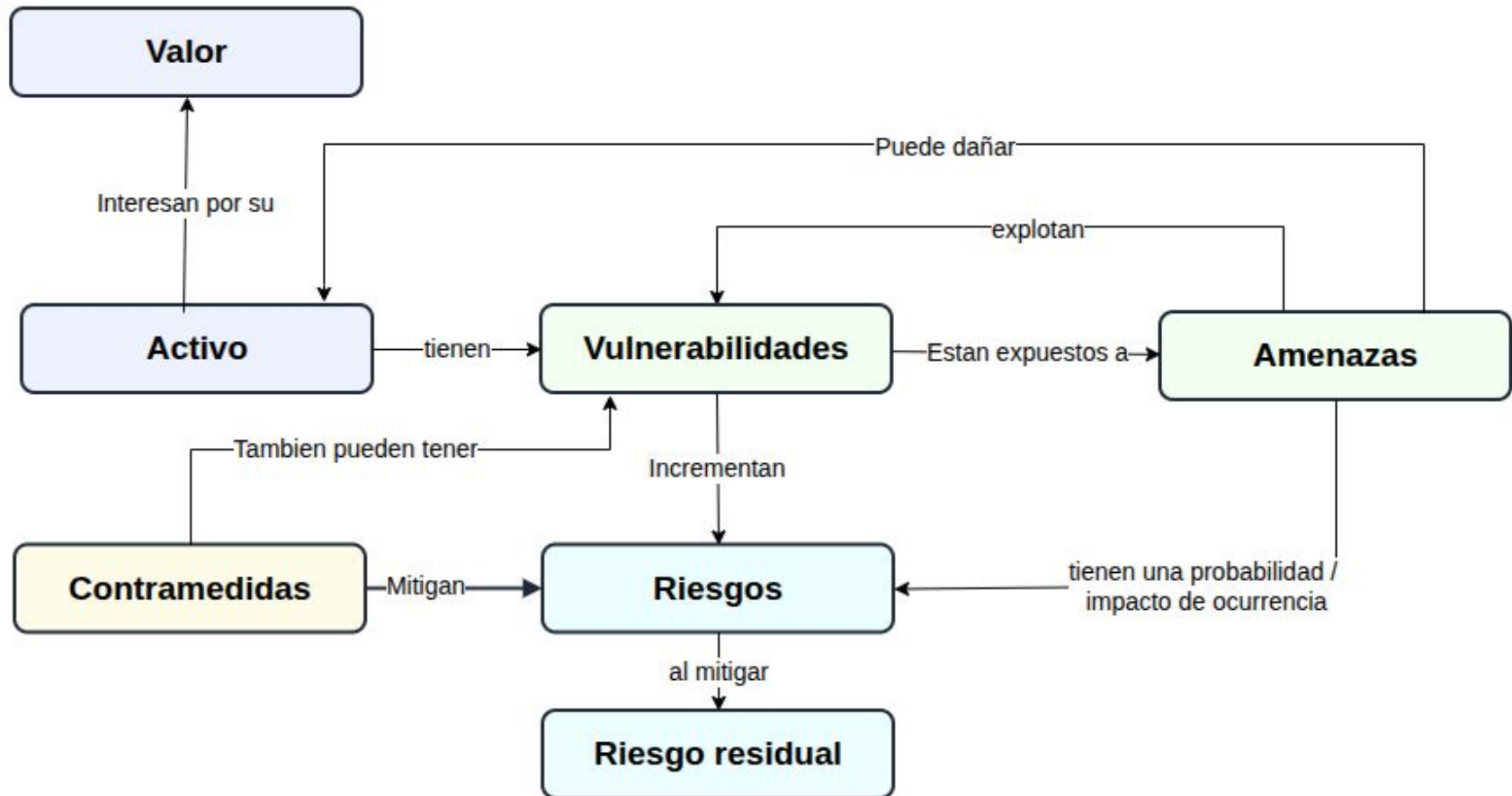
Repaso

Cualquier circunstancia o evento con el potencial de afectar negativamente

- las operaciones,
- activos de la organizacion
- individuos por accesos no autorizados
- destruccion divulgación,
- alteración de la información
- y/o denegacion de servicio.



Riesgo - Modelo conceptual de riesgo en SI



Repaso

- **Seguridad por diseño**

La seguridad es una propiedad estructural del sistema, definida principalmente en decisiones de requerimientos y arquitectura.

- **Superficie de ataque**

Es la consecuencia **estructural** directa de esas decisiones: interfaces, funcionalidades, integraciones y dependencias que quedan expuestas.

- **Defensa en profundidad**

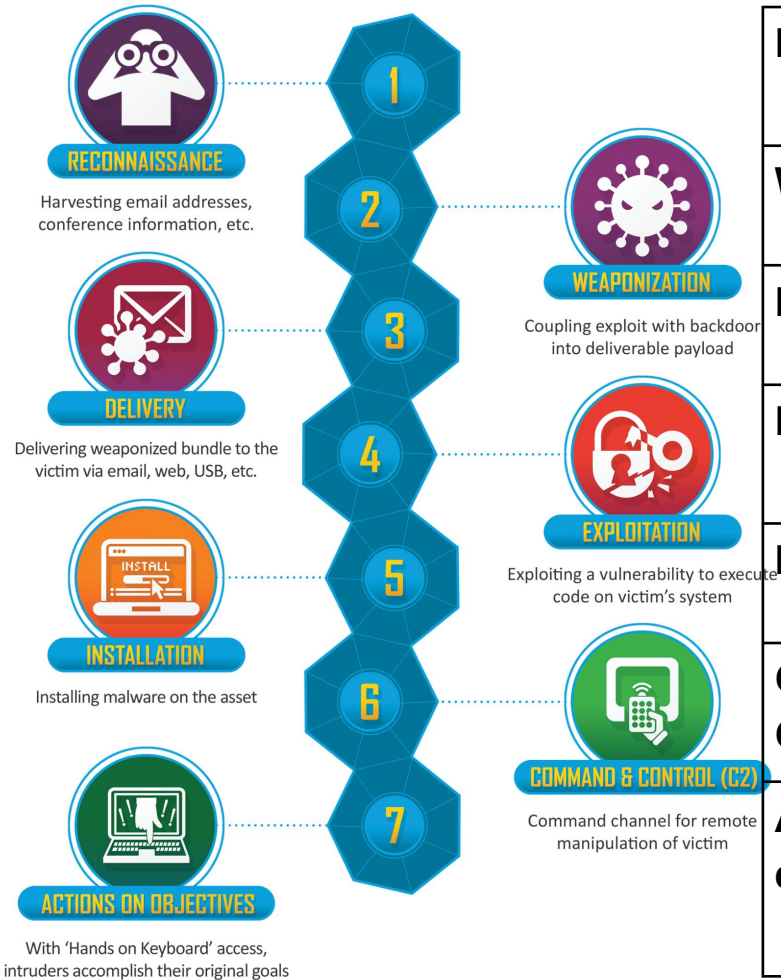
Asume que esa superficie **siempre existe** y que los controles pueden fallar, por lo que no se depende de un único mecanismo para proteger el sistema.

**Primero diseñamos reduciendo al máximo la exposición,
y recién ahí ponemos controles.**

Repaso



Cyber Kill Chain (Lockheed Martin)



Reconocimiento	Recolección de información sobre el objetivo y su entorno
Weaponization	Preparación del exploit y del payload
Entrega	Envío del vector de ataque
Explotación	Explotación de una vulnerabilidad para ejecutar código o ganar acceso.
Instalación	Despliegue y persistencia
Comando y Control (C2)	Establecimiento de comunicación con la infraestructura del atacante.
Acciones sobre el objetivo	Movimiento lateral, escalamiento de privilegios, robo, alteración o destrucción de información.

Repaso



Las herramientas son ilustrativas y pueden cumplir distintos roles según el contexto.

Repaso

HAY VULNERABILIDADES
NUESTRO SISTEMA VA A SER ATACADO
ALGÚN ATAQUE FUNCIONARÁ.

CADA DECISIÓN CAMBIA
EL NIVEL DE SEGURIDAD
DE NUESTRA APLICACIÓN

AUN SI NO TENEMOS EN CUENTA
SU SEGURIDAD EN LA DECISIÓN

- Que la seguridad no aparezca después del diseño o la implementación.
- Que quede plasmada en el diseño y se contemple durante todo el ciclo de vida.
- Comprender que muchas amenazas no nacen de errores de implementación, sino de decisiones arquitectónicas tomadas mucho antes.
- Entender que las estrategias y prioridades organizacionales, o la ausencia de las mismas, impactan en las decisiones de arquitectura y en su seguridad.

amenazas ↔ arquitectura ↔ organización

**Una debilidad de seguridad puede surgir
no solo por una vulnerabilidad técnica,
sino también por una decisión
arquitectónica y operativa equivocada.**

- Comprender cómo un sistema podría ser abusado, manipulado o **utilizado de manera no prevista**.
- Analizar el sistema desde la **perspectiva de un atacante**, considerando objetivos, capacidades y oportunidades.
- Identificar **superficies de ataque**, relaciones de confianza y **decisiones** arquitectónicas que puedan **introducir riesgo**.
- Anticipar **escenarios plausibles de abuso** antes de la implementación o despliegue.
- **Pensar como atacante** no implica atacar un sistema, sino comprender mejor cómo **defenderlo**.

Para gestionar eficientemente el riesgo para la seguridad de la información que maneja nuestro sistema, es fundamental analizar:

- Cómo circula la información por el sistema (su arquitectura)
- En qué puntos hay amenazas que hacen posible vulnerar la seguridad
- Qué capacidades tendría un potencial atacante
- De qué manera se puede disminuir ese riesgo con controles técnicos u organizacionales

**La arquitectura define el flujo de información,
permite anticipar amenazas y definir controles adecuados.**

Referencias

Respuesta a crisis de confianza y fraude sistémico

Referencias

https://owasp.org/www-community/Threat_Modeling_Process

<https://www.udemy.com/course/modelado-de-amenazas>

https://safecode.org/wp-content/uploads/2017/05/SAFECode_TM_Whitepaper.pdf

TESIS DE

<https://repository.usta.edu.co/bitstream/handle/11634/44064/2022andresmelo.pdf>

El NIST define al modelado de amenazas como

Método que capta el funcionamiento del sistema para identificar y entender las amenazas potenciales, los objetivos y tácticas de los agentes de amenazas

Para establecer controles de seguridad que permitan mitigarlos.

Con el propósito de captar la esencia del funcionamiento de un sistema

Se suele analizar la arquitectura de la aplicación

Realizar diagramas de flujo de datos que destaquen de manera granular cómo cada componente o servicio interactúa entre sí

Analizar cuáles vulnerabilidades podrían ser aprovechadas por las amenazas

<https://csrc.nist.gov/pubs/sp/800/154/ipd>

NIST.IR.8397 Directrices de Verificación de SW

Information Technology Laboratory

EXECUTIVE ORDER 14028, IMPROVING THE NATION'S CYBERSECURITY

Software Supply Chain Security Guidance

Software Security in Supply Chains

Software Cybersecurity for Producers and Purchasers

Critical Software

Software Verification

Introduction

Background & Approach

Recommended Minimum Standard for Vendor or Developer Verification of Code

FAQs

Cybersecurity Labeling for Consumers

Workshops & Call for Papers

News & Updates

Engage

Fact Sheet

Recommended Minimum Standard for Vendor or Developer Verification of Code

[f](#) [in](#) [tw](#) [envelope](#)

The following are recommended minimums for verification of code by developers. Some of the minimums are the precursors to effective testing and some are the logical outcomes of the testing. Each of the techniques is described in the accompanying [document](#).

Technique Class	Technique	Description & Reference to Recommended Minimums Document
Threat modeling	Threat modeling helps identify key or potentially overlooked testing targets.	Section 2.1. Threat modeling methods create an abstraction of the system, profiles of potential attackers and their goals and methods, and a catalog of potential threats. Threat modeling

<https://nvlpubs.nist.gov/nistpubs/ir/2021/NIST.IR.8397.pdf>

<https://www.nist.gov/itl/executive-order-improving-nations-cybersecurity/recommended-minimum-standard-vendor-or-developer>

2	Recommended Minimum Standard for Developer Testing	4
2.1	Threat Modeling	5
2.2	Automated Testing	6
2.3	Code-Based, or Static, Analysis	6
2.4	Review for Hardcoded Secrets	7
2.5	Run with Language-Provided Checks and Protection	7
2.6	Black Box Test Cases	7
2.7	Code-Based Test Cases	8
2.8	Historical Test Cases	8
2.9	Fuzzing	8
2.10	Web Application Scanning	8
2.11	Check Included Software Components	9

Tipo	Técnica
Modelado de amenazas	Ayuda a identificar objetivos clave o potencialmente pasados por alto para las pruebas o el diseño.
Pruebas automatizadas	A medida que las pruebas se automatizan, se pueden repetir con frecuencia, por ejemplo, en cada confirmación o antes de que se cierre un problema.
Análisis basado en código (estático)	Utilice un escáner de código para buscar los principales errores.
	Buscar credenciales y claves en el código.
Análisis dinámico (es decir, ejecutar el programa en casos de prueba):	Ejecutar con controles y protecciones integrados.
	Crear casos de prueba de "caja negra".
	Crear casos de prueba estructurales (core) basados en el código.
	Utilizar casos de prueba creados para atrapar errores que se tuvieron antes.
	Ejecutar un "fuzzer" (Herramienta para probar la resistencia del software a entradas inesperadas).
	Si el software puede estar conectado a Internet, ejecutar un escáner de aplicaciones web.
Verificar bibliotecas	Utilizar técnicas similares para asegurarse de que las bibliotecas, paquetes, servicios, etc. incluidos no sean menos seguros que su propio código.
Corregir errores	Corregir los errores críticos que se descubran.

Metodología STRIDE

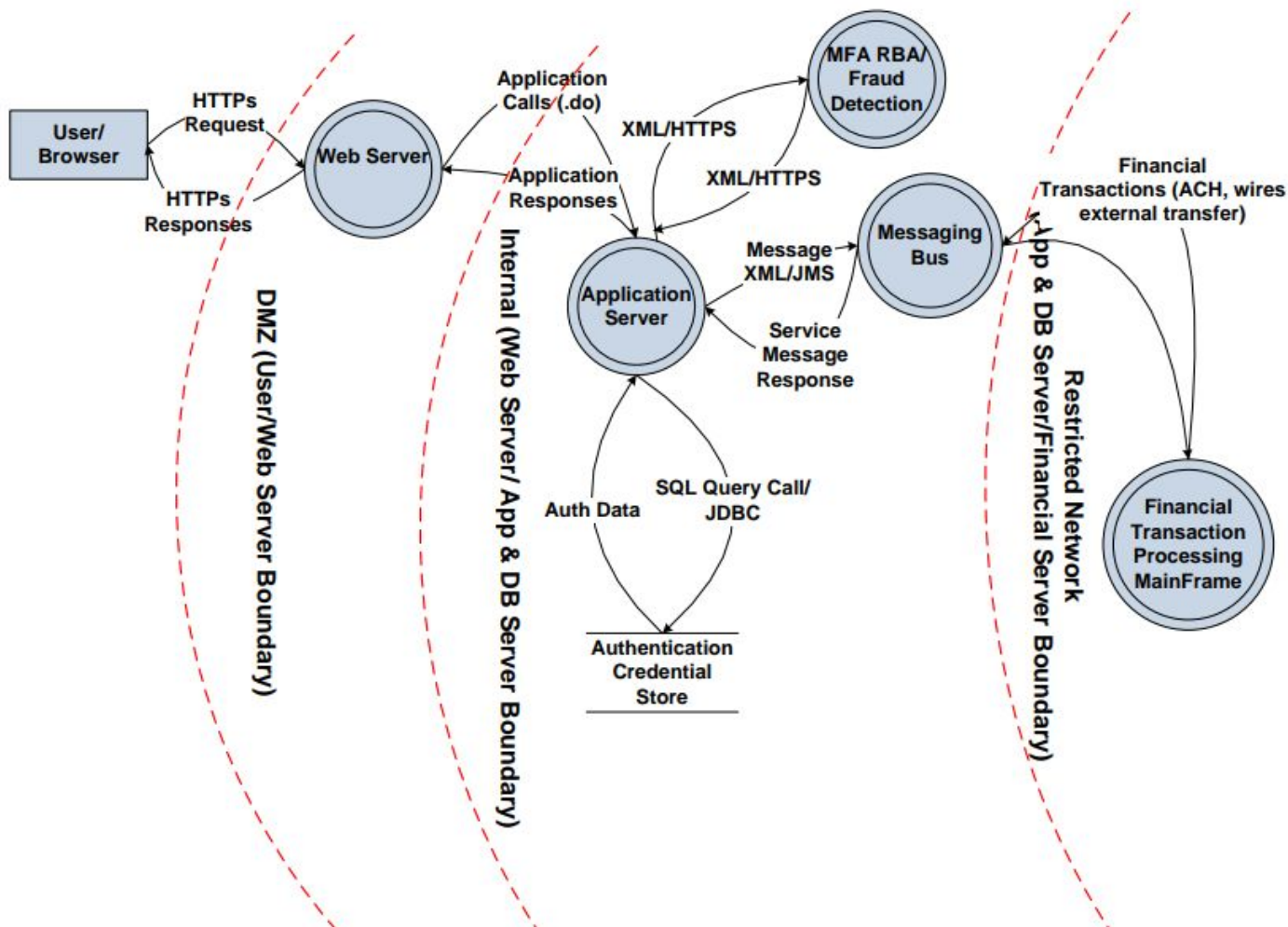
Respuesta a crisis de confianza y fraude sistémico

Queremos responder:

- ¿En que estamos trabajando?
- ¿que modulos o partes forman mi sistema?
- ¿Qué puede ir mal?
- ¿Qué vamos a hacer al respecto?
- ¿Hicimos un trabajo suficientemente bueno?

**Abordar las amenazas potenciales antes,
lo que significa que los desarrolladores no tienen que volver atrás a rediseñar
para corregir las vulnerabilidades**

Análisis de amenazas: Diagramas de flujo de datos (I)



Análisis de amenazas: Diagramas de flujo de datos (II)

	Nombre	Descripción
	Entidad Externa	Se utiliza para representar cualquier entidad fuera de la aplicación.
	Proceso	Representa una tarea que manipula datos dentro de la aplicación. procesar los datos o ejecutar una acción con ellos.
	Proceso Múltiple	Para presentar una colección de subprocessos. Puede descomponerse en sus subprocessos en otro DFD.
	Almacén de Datos	Se utiliza para representar ubicaciones donde se almacenan datos.
	Flujo de Datos	Representa el movimiento de datos dentro de la aplicación. La dirección está representada por la flecha.
	Frontera de Confianza	Para representar el cambio de niveles de confianza a medida que los datos fluyen a través de la aplicación.

Figure 1: Data Flow Diagram for the College Library Website.

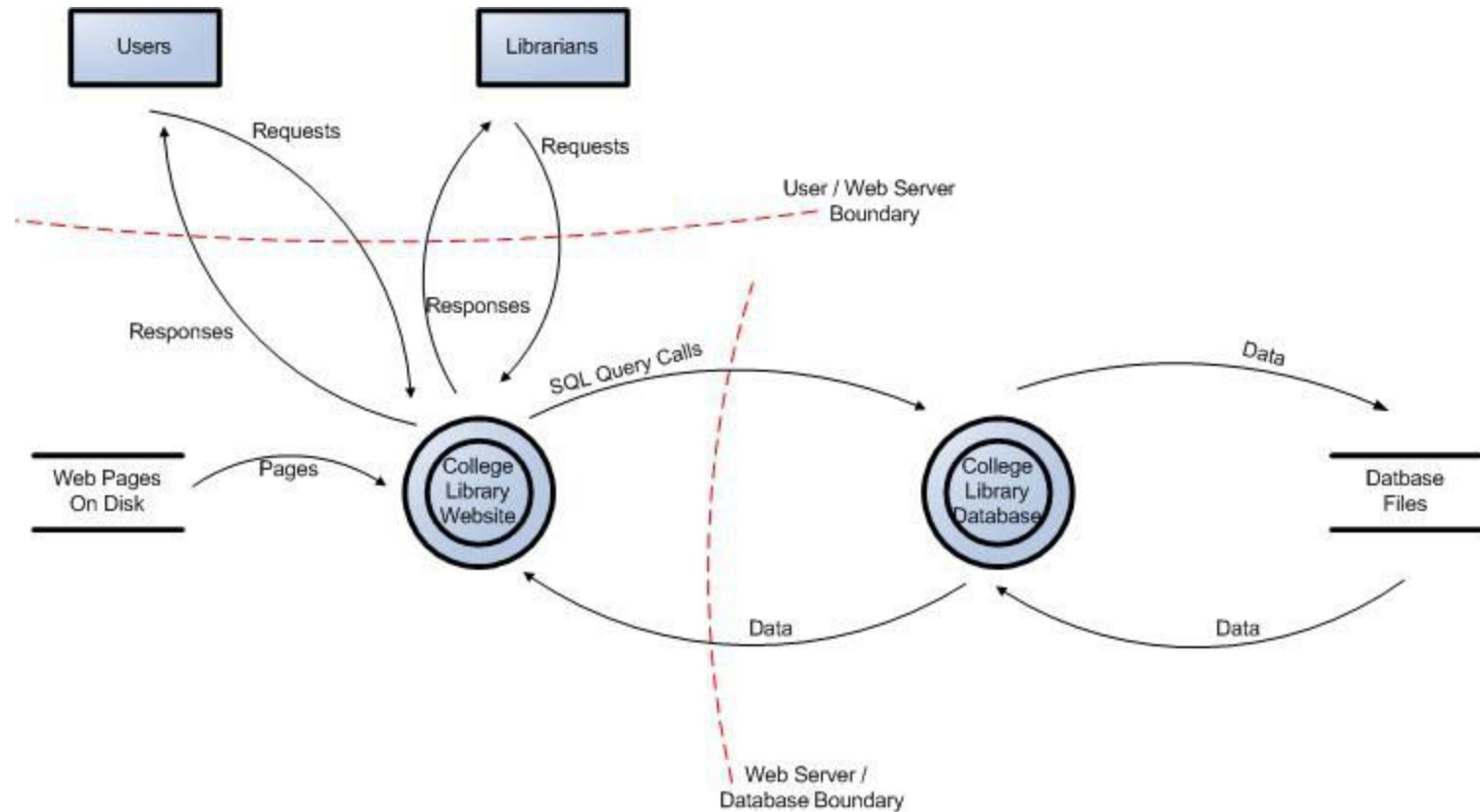
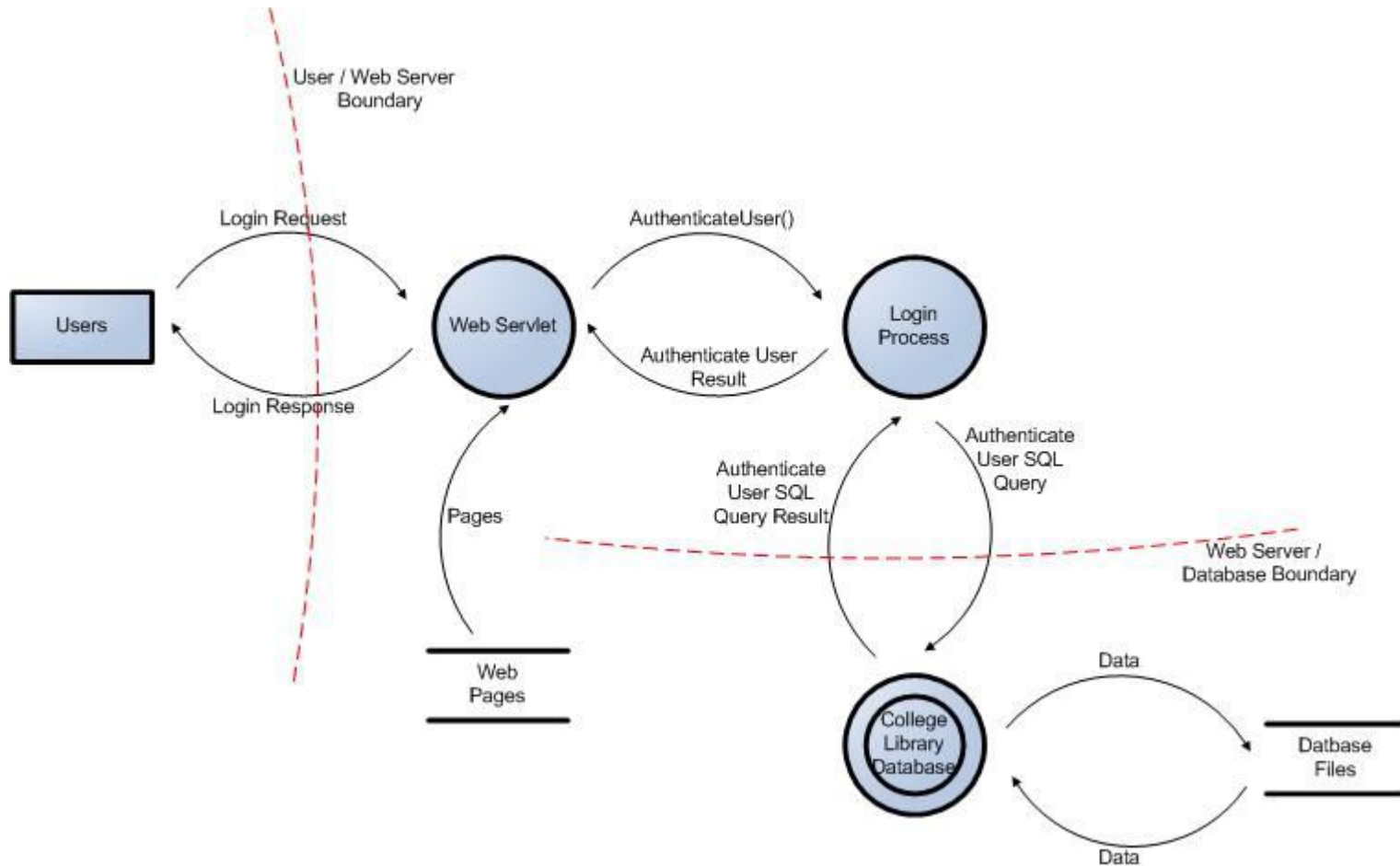


Figure 2: User Login Data Flow Diagram for the College Library Website.



Cada punto de un sistema en el que la información pasa de un usuario o proceso a otro, o de un nivel de privilegios a otro, es una frontera de confianza.

Ejemplos:

- Llamadas al sistema
- Usuario-browser
- Browser-internet
- Internet-Servidor Web
- Servidor Web-Base de Datos

En cada uno de estos puntos puede concebiblemente producirse un ataque

STRIDE

S poofing	Un [usuario proceso host] se hace pasar por otro.	Autenticación
T ampering (Manipulación)	Alguien realiza cambios sin autorización en alguna parte de un sistema. (maliciosamente).	Integridad
R epudiation	Realizar operaciones prohibidas en un sistema que carece de la capacidad de rastrear dichas operaciones. Ej poder negar una acción realizada en un web	No repudio
I nformation Disclosure	Acceder a información a la que no está autorizado. Ej. Leer datos en tránsito.	Confidencialidad
D enial of Service	Alguien consigue denegar el acceso al servicio a usuarios autorizados. Ej. que un servidor web no esté disponible.	Disponibilidad
E levation of Privilege	Elevar los privilegios con que accede sin estar autorizado. Ej. Acceso a Información confidencial	Autorización

- Modificación de los logs del sistema para borrar huellas
 - Modificación de la base de datos sin autorización
 - Interferencia del tráfico de red
 - Cambios no autorizados en configuraciones o datos
 - El tampering se suele combatir con
 - Hashes encadenados en los logs
 - Restricciones de escritura en los logs, especialmente en las bases de datos
-

El repudio es un caso un poco especial porque “se sale” del software: ocurre en la capa de negocio.

El combate a los ataques de repudio es

- Usando logs, que tienen que
 - existir y ser razonablemente exhaustivos, y
 - estar protegidos contra tampering
- Usando mecanismos contra spoofing,
 - Tener una seguridad razonable de que la persona es la única que opera en su nombre.
- Usando mecanismos criptográficos como Firma Digital

- Lograr entrar como un usuario legítimo y acceder a sus datos.
- Pedir datos de otro por fallos de autorización.
- Por un fallo de implementación permite acceder a datos sin autorización.
- Un administrador revisa datos privados de usuarios sin permiso.

Formas de combatirlo:

- Tener métodos fuertes de autenticación
- Verificar que la autorización esté correctamente implementada
- Verificar que las entradas externas se validen exhaustivamente
- Asegurarse de que los logs del sistema sean robustos también contra el administrador. Y no tengan información de mas.

- La red se satura de consultas
- Los servidores se saturan de consultas
- Los recursos del sistema se saturan de consultas
- El sistema se cae por una entrada maliciosa

Formas de combatirlo:

- Recursos elásticos: escalabilidad y alta disponibilidad
 - Hacer que falsificar una consulta requiera trabajo
 - Validar siempre las entradas
-

Modelo STRIDE: Elevation of Privilege

- Alguien aprovecha una vulnerabilidad
 - en el kernel para otorgarse privilegios a sí mismo (exploits)
 - Malware aprovecha permisos excesivos de un servicio.
- Alguien “engaña” a un programa que no valida entradas.
 - Ej. Una aplicación web permite acceder a funciones de administrador cambiando un parámetro.
- Un empleado accede al panel de administración sin autorización por mala configuración.

Formas de combatirlo:

- Mínimo Privilegio
- Control de acceso
- MFA
- Parches de seguridad
- Auditoría (detección)
- Hardening

El modelo STRIDE se usa de la siguiente manera:

- Se realiza un DFD de nuestro sistema.
- Se busca cómo podría realizarse un ataque de cada tipo en cada elemento.
- Fundamentalmente en los flujos de datos que cruzan fronteras de confianza.

Para cada uno de estos riesgos, hay que decidir si:

- Mitigarlo
- Eliminarlo
- Asumir el riesgo
- Transferir el riesgo

Modelo STRIDE

Alternativamente, se pueden buscar las amenazas más comunes para cada tipo de elemento:

Elemento	S Suplantación	T Manipulación	R Repudio	I Divulgación	D Denegación	E Elevación
Entidad externa	X		X			
Proceso	X	X	X	X	X	X
Flujo de datos		X		X	X	
Almacén de datos		X	?	X	X	

- Después de revisar cada elemento y mitigar los riesgos asociados
- Es recomendable actualizar el DFD
 - con los cambios adoptados y volver a
 - repetir el proceso:
 - termina cuando los riesgos se vuelven aceptables por la dificultad de realizar un ataque.
- Output
 - Lista estructurada de amenazas
 - Relación con impacto / riesgo (ver DREAD mas adelante)
 - Insumo para diseño/arquitectura posterior

Una vez enumerados los elementos, se busca qué amenazas podrían existir de los tipos correspondientes

1. Cliente Web: Elemento externo -> Spoofing y Repudio.
 - a. Spoofing de un usuario: Se evita utilizando un mecanismo de autenticación.
 - b. Repudio de acciones: Se mitiga registrando las acciones de los usuarios, preferentemente con detalles adicionales como hora e IP de conexión.
Auditorias, Firma de las Acciones, MFA.
2. Flujo de datos cliente - Front End: Flujo de Datos -> Tampering, Information Disclosure y Denial of Service.
 - a. Tampering: Utilizar TLS garantiza integridad
 - b. Information Disclosure: Utilizar TLS garantiza confidencialidad
 - c. Denial of Service:
 - i. En el contexto del flujo de datos, sería una congestión del canal, u otros ataques que no está en manos del software en cuestión.
 - ii. La congestión se soluciona (por ejemplo) con escalabilidad de recursos, o haciendo que los clientes muestren un autenticador para las consultas.

Herramienta OWASP Threat Dragon

Descargas en



<https://github.com/OWASP/threat-dragon/>

Online en <https://www.threatdragon.com>



Threat Dragon v2.6.1-latest English ▾ Logged in as local-user

Generic CMS

Generic threat model for a Content Management System (CMS)
provided by the OWASP Threat Model Cookbook:
threat-model-cookbook/Flow Diagram/generic-cms

```
graph TD
    User[Generic / Privilege user] -- HTTPS --> WebServer((Web server))
    subgraph Internet
        WebServer -- Unsecured MySQL connection --> Admin[Admin]
        WebServer -- Unsecured MySQL connection --> Database[Database]
    end
    WebServer -- Push to Bucket --> CDN((CDN network))
    CDN -- HTTP --> User
```

Entities

Components

Process

Store

Actor

Data Flow

Boundaries

Trust Boundary

Metadata

Descriptive text

Properties

Threats

Select an element on the graph to edit

Select an element on the graph to add a threat

Export ▾ Close Save

*El rango de lo que pensamos y hacemos
está limitado por aquello de lo que no nos damos cuenta.*

*Y es precisamente el hecho de no darnos cuenta
de que no nos damos cuenta
lo que impide
que podamos hacer algo
por cambiarlo.*

*Hasta que nos demos cuenta
de que no nos damos cuenta
seguirá moldeando nuestro pensamiento y nuestra acción.*

R. D. Laing

Lo que es importante, rara vez es urgente. Y lo que es urgente rara vez es importante.

Ewight Eisenhower

Evaluación del riesgo modelo DREAD

Tipo	Descripción
D amage	Daño o impacto que tiene la explotación de la amenaza a causa de la vulnerabilidad.
R eproducibility	Reproducción del incidente, probabilidad de repetirse el daño.
E xploitability	Explotación de la vulnerabilidad, qué tan complejo es mitigarla y qué costos están asociados a este proceso.
A ffected Users	Determinar el impacto de afectación del incidente sobre qué cantidad de usuarios y recursos aplicados.
D iscoverability	Grado de facilidad de exposición y descubrimiento de la vulnerabilidad.

Evaluación del riesgo modelo DREAD

Se evalúa cada categoría con un valor de 0 a 10
El riesgo es la suma dividido 5

Tipo	Descripción
D amage	0 indica ningún daño y 10 representa una catástrofe
R eproducibility	0 indica que es muy difícil de explotar y 10 que es muy fácil
E xploitability	0 significa que es muy fácil de mitigar 10 indica que es difícil de mitigar.
A ffected Users	0 no afecta a ningún usuario, 10 afecta a muchos usuarios o sistemas. (o usuarios clave como root)
D iscoverability	0 significa que es difícil de descubrir, 10 indica que es muy fácil de detectar.

Manifesto



THREAT MODELING MANIFESTO

What is threat modeling?

Threat modeling is analyzing representations of a system to highlight concerns about security and privacy characteristics.

At the highest levels, when we threat model, we ask four key questions:

1. What are we working on?
2. What can go wrong?
3. What are we going to do about it?
4. Did we do a good enough job?

Análisis estimado de campus - Guarani

Respuesta a crisis de confianza y fraude sistémico

Implementar un modelo de amenazas de la arquitectura estimada del campus y guarani de FIUBA.

Si faltan datos asumanlos razonablemente.

Ej, si no saben que base de datos usa, luego de hacer un poco de inteligencia, la inventan.

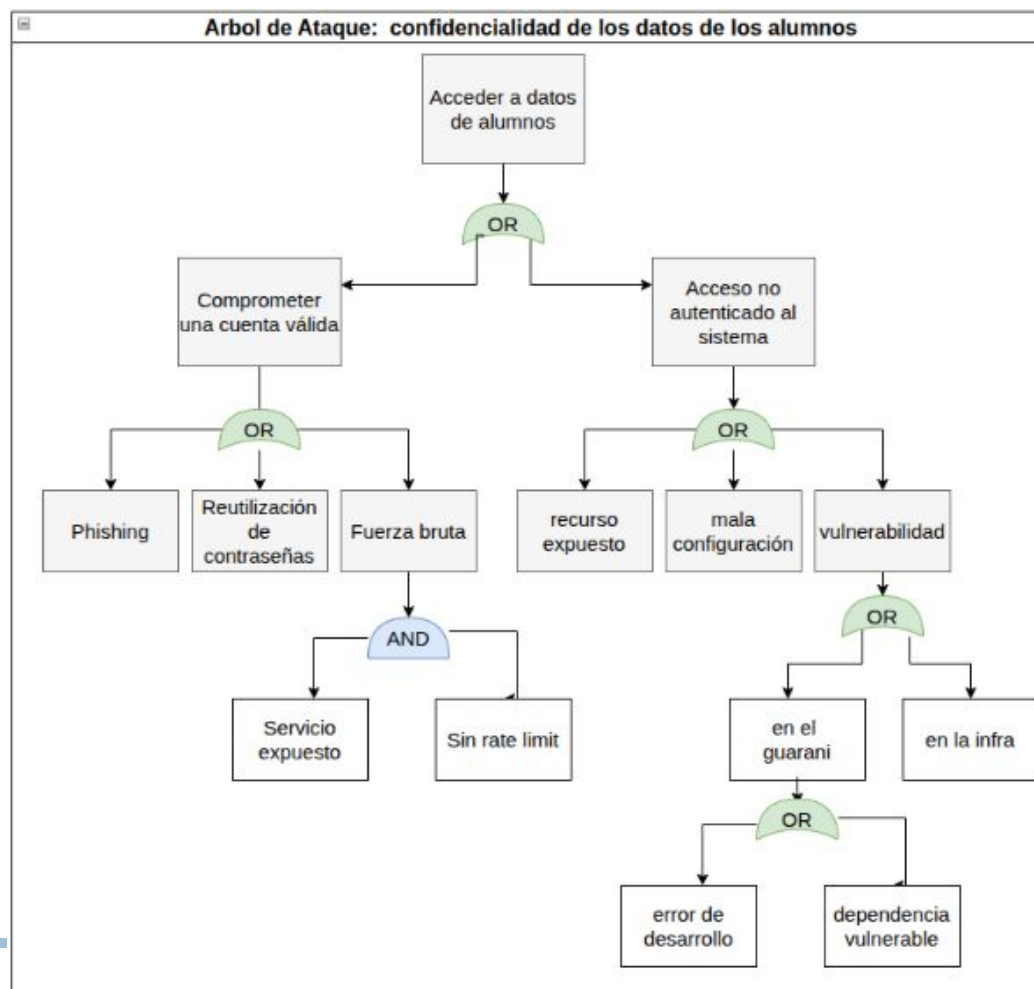
- Identificación de Actores y Datos Sensibles
 - Estimación del DFD
 - **Identificación de Amenazas:** Identificar posibles amenazas a la seguridad
 - **Evaluación de Amenazas:** Si es posible, incluir detalles como las vulnerabilidades, los activos comprometidos y los posibles impactos.
 - **Evaluar riesgo usando DREAD:** Asignar una valoración de riesgo a las amenazas (por lo menos una).
 - **Propuesta de Contramedidas:** proponer contramedidas de seguridad adecuadas. (Puede ser un rediseño de la arquitectura)
 - **Presentación y Documentación:** informe que resuma su análisis de amenazas y las contramedidas propuestas (hasta 5 paginas).
-

Otros Modelos complementarios

Respuesta a crisis de confianza y fraude sistémico

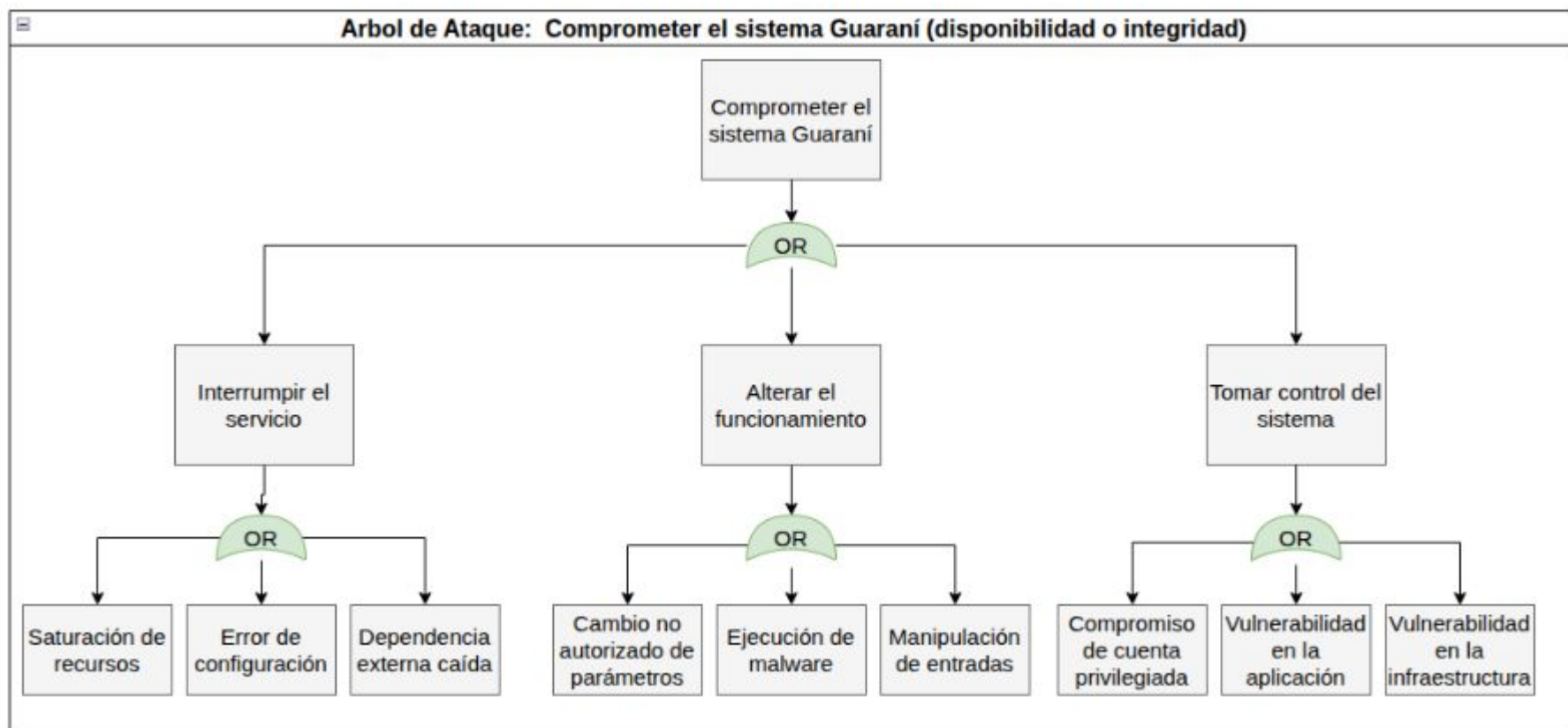
Arboles de ataque

El diagrama representa distintos caminos posibles mediante los cuales un atacante podría acceder a información de alumnos, sin pretender ser exhaustivo.

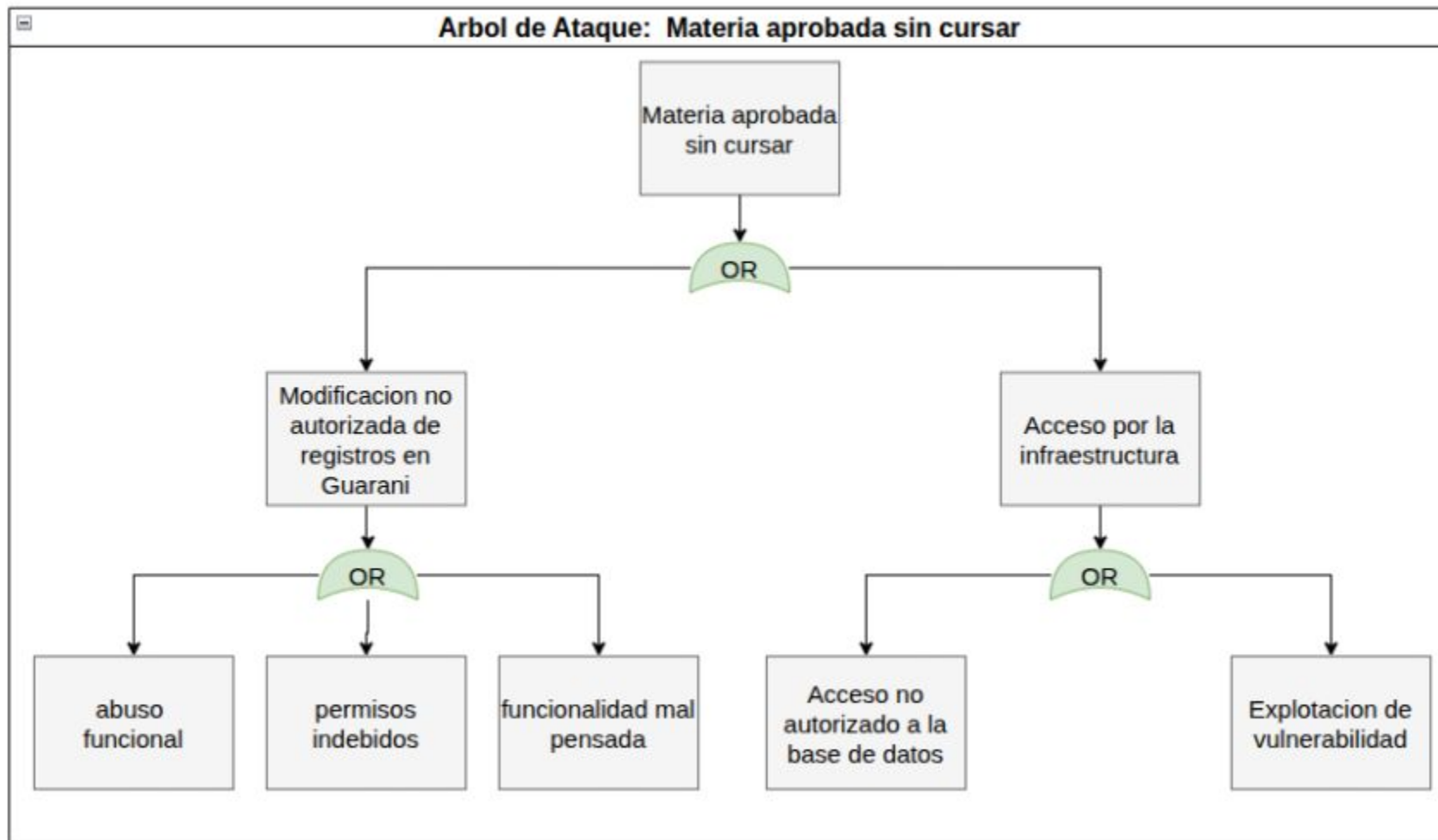


Arboles de ataque: Sistema guaraní

El diagrama representa distintos caminos posibles (inventados) mediante los cuales un atacante puede actuar sobre el guaraní.



Árbol de ataque sobre el proceso académico



A partir de situaciones cercanas al entorno académico, se invita a reflexionar sobre la relación entre datos, sistemas y procesos, y sobre cómo decisiones aparentemente correctas pueden producir resultados incorrectos.

Preguntas para pensar los límites del enfoque técnico

- ¿Un pentest detectaría que una materia fue aprobada fuera de término?
- ¿Qué problemas no aparecen en un análisis técnico clásico?
- ¿Todo lo importante en seguridad se puede automatizar?

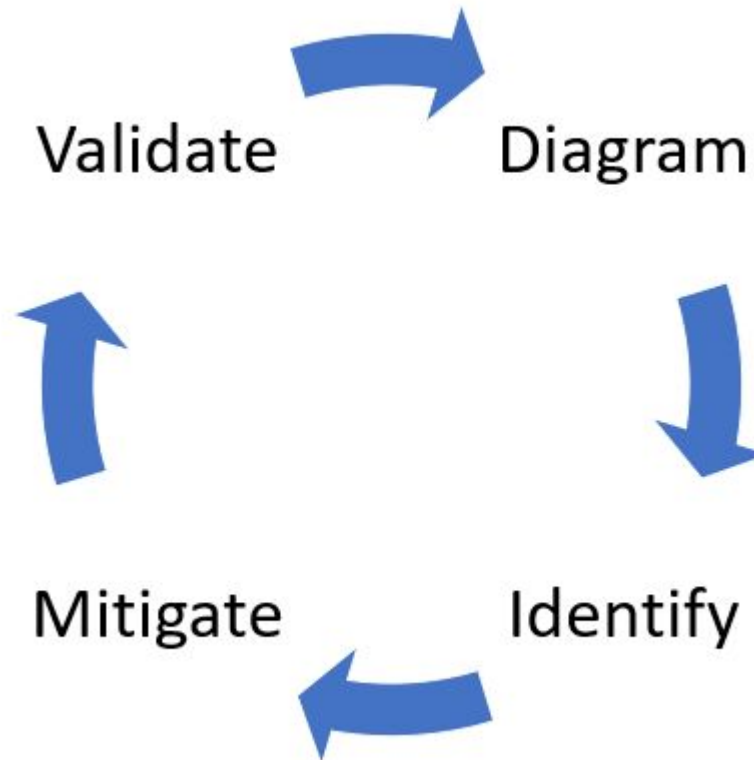
Preguntas para reflexionar sobre decisiones e impacto

- ¿Es más grave un dato incorrecto o una decisión basada en un dato incorrecto?
- Si los datos están correctos, ¿puede el resultado ser incorrecto?

Preguntas ancladas en Guaraní

- ¿Qué pasaría si mañana aparece una materia aprobada que nunca cursaste?
 - ¿Qué es peor: que alguien lea una nota o que una nota cambie sin que nadie lo note?
 - ¿Eso es un problema de datos, de sistema o de proceso?
 - ¿Quién debería detectar algo así: el sistema, una persona o un control?
-

Proceso de MA: Microsoft



Es un Modelo complementario dividido en fases



Proceso de MA: Fases del Modelo PASTA



Modelado de Amenazas en Privacidad: LINDUN

LINDUN es un modelo de amenazas orientado a la privacidad de los datos, no tanto a la seguridad en general.

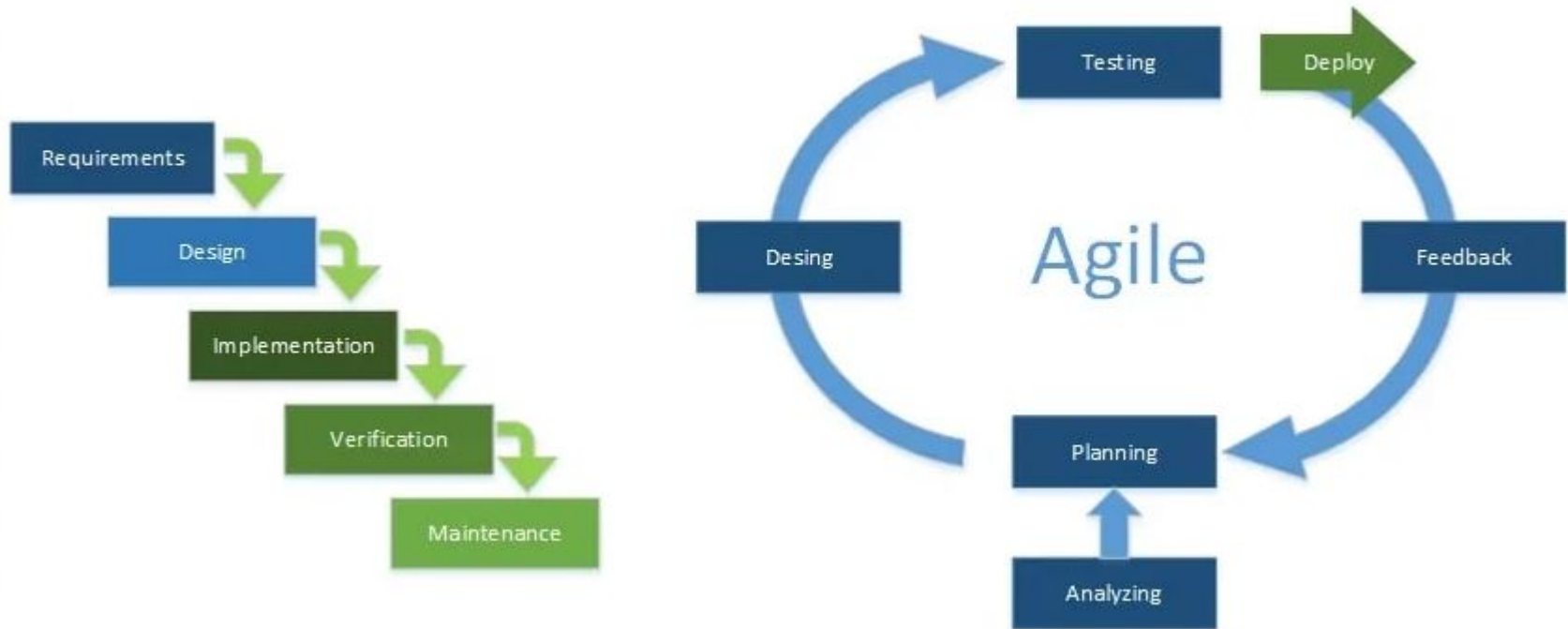
L inkability	Vinculación de datos: Posibilidad de relacionar distintos datos, acciones o eventos con un mismo usuario o entidad.
I dentifiability	Capacidad de identificar de manera directa o indirecta a una persona a partir de los datos disponibles.
N on-repudiation	No repudio: Capacidad de demostrar que una acción o transacción fue realizada por una entidad específica y no puede ser negada.
D etectability	Detectabilidad: Posibilidad de detectar la existencia de datos, actividades, comunicaciones o eventos sensibles.
D ata Disclosure	Divulgación de datos: Exposición o acceso no autorizado a información sensible o privada.
U nawareness	Falta de conocimiento: Situación en la que los usuarios desconocen cómo se recopilan, utilizan o comparten sus datos.
N on-compliance	Incumplimiento normativo: Falta de cumplimiento de políticas, regulaciones, leyes o requisitos de privacidad y seguridad.

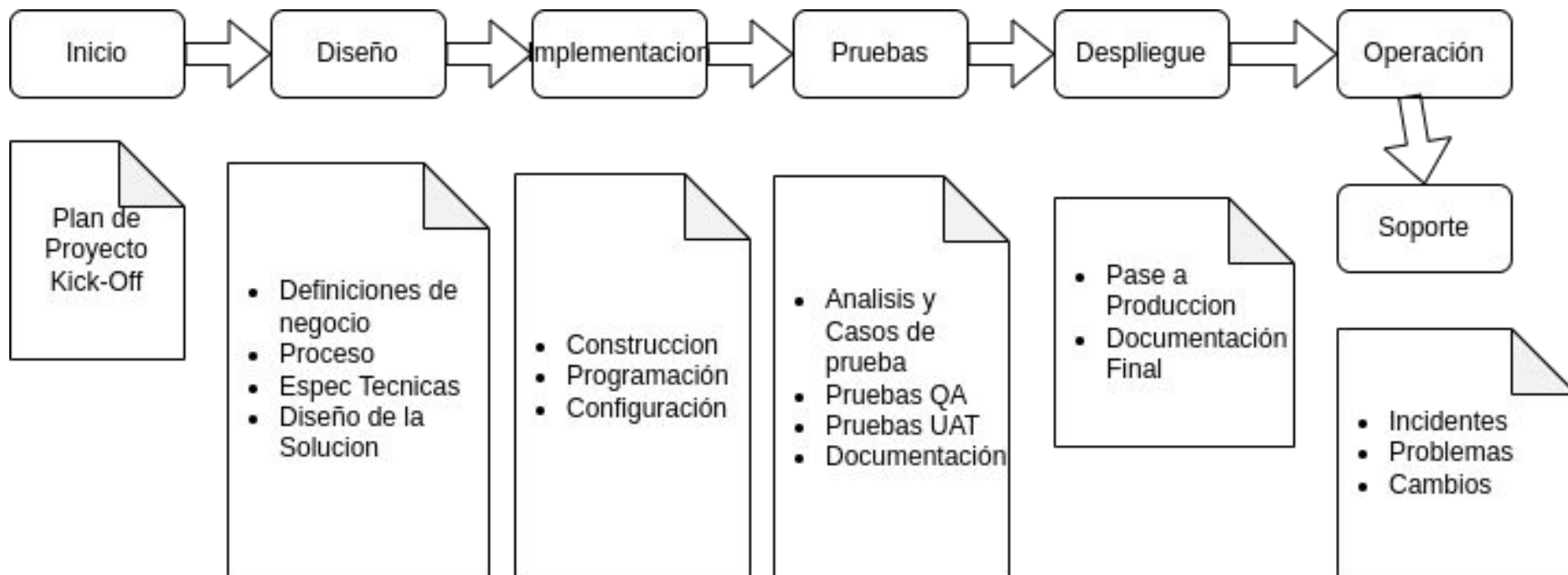
Aspecto	STRIDE	LINDDUN
Enfoque principal	Seguridad (ataques, vulnerabilidades)	Privacidad de datos
Cuándo se usa	Evaluar amenazas de seguridad comunes	Evaluar cómo el diseño afecta la privacidad
Tipo de amenazas	Spoofing, DoS, manipulación, etc.	Linkability, identifiability, datos sin consentimiento
Más útil si	Tenes que proteger la seguridad global del sistema	Necesitas cumplir requisitos de privacidad y protección de datos

Seguridad desde el diseño

Respuesta a crisis de confianza y fraude sistémico

Waterfall vs. Agile





En el diseño

Ubicar los controles de seguridad y/o las variables de una aplicación web únicamente en el lado del cliente (navegador), lo cual puede ser manipulado fácilmente por un usuario malintencionado.

Omitir definir quiénes NO deben tener acceso a cierta información, lo cual puede llevar a brechas de seguridad al no restringir adecuadamente el acceso.

En la implementación

Errores al programar

No validar entradas: Construir consultas SQL directamente con entradas de texto del usuario sin realizar validaciones previas, lo que puede exponer la aplicación a ataques de inyección SQL.

Incorrecto manejo excepción: Manejar excepciones simplemente capturándolas e ignorando los errores subyacentes, lo que puede ocultar problemas críticos y afectar la estabilidad de la aplicación.

En el despliegue:

Configuración insegura de servidores y servicios: Desplegar aplicaciones con configuraciones por defecto o inseguras, lo que puede dejar expuestos servicios críticos a ataques externos.

Falta de protocolos seguros: No utilizar protocolos de comunicación seguros como HTTPS, lo que expone los datos transmitidos a interceptaciones y modificaciones.

En la operación:

Monitoreo insuficiente: No implementar sistemas de monitoreo efectivos que permitan detectar y responder a actividades sospechosas o ataques en tiempo real.

Actualizaciones y parches desatendidos: No mantener actualizados los sistemas y aplicaciones, lo que puede dejar vulnerabilidades sin corregir y explotables por atacantes.

En el soporte:

Gestión deficiente ante incidencias: No tener un proceso efectivo para manejar incidentes de seguridad, lo que puede resultar en respuestas lentas o inadecuadas ante brechas o fallos de seguridad.

Capacitación insuficiente del personal de soporte: No capacitar adecuadamente al personal de soporte en prácticas de seguridad, lo que puede llevar a errores en la gestión de configuraciones y datos sensibles.

Durante la Desactivación de Sistemas o Software

Eliminación incompleta de datos: No asegurarse de que todos los datos sensibles almacenados por el sistema sean completamente eliminados o encriptados. Esto puede dejar información confidencial expuesta a posibles filtraciones.

Mantenimiento de accesos residuales: Dejar accesos activos o cuentas de usuario sin desactivar completamente, lo que puede permitir entradas no autorizadas a sistemas aún en línea o a otros sistemas interconectados.

Desatención de copias de seguridad: No gestionar adecuadamente las copias de seguridad que contienen datos del sistema desactivado, pudiendo ser una fuente de riesgos si estas copias caen en manos equivocadas.

Durante la Desactivación de Sistemas o Software

- Falta de documentación de la desactivación:** No mantener un registro detallado de las acciones de desactivación, incluyendo qué, cómo y cuándo se desactivó. La falta de esta documentación puede llevar a confusiones o a la reactivación inadvertida de componentes del sistema que se suponían desactivados.
 - No revisar dependencias:** Ignorar cómo la desactivación de un sistema podría afectar a otros sistemas interdependientes. Esto puede causar interrupciones inesperadas o fallos en otros servicios que dependen del sistema desactivado.
 - Seguridad perimetral olvidada:** No asegurar adecuadamente el entorno físico o virtual donde residía el sistema, dejando vulnerabilidades que podrían ser explotadas aun después de que el sistema está oficialmente "fuera de servicio".
-

Seguridad desde el principio y todo el tiempo

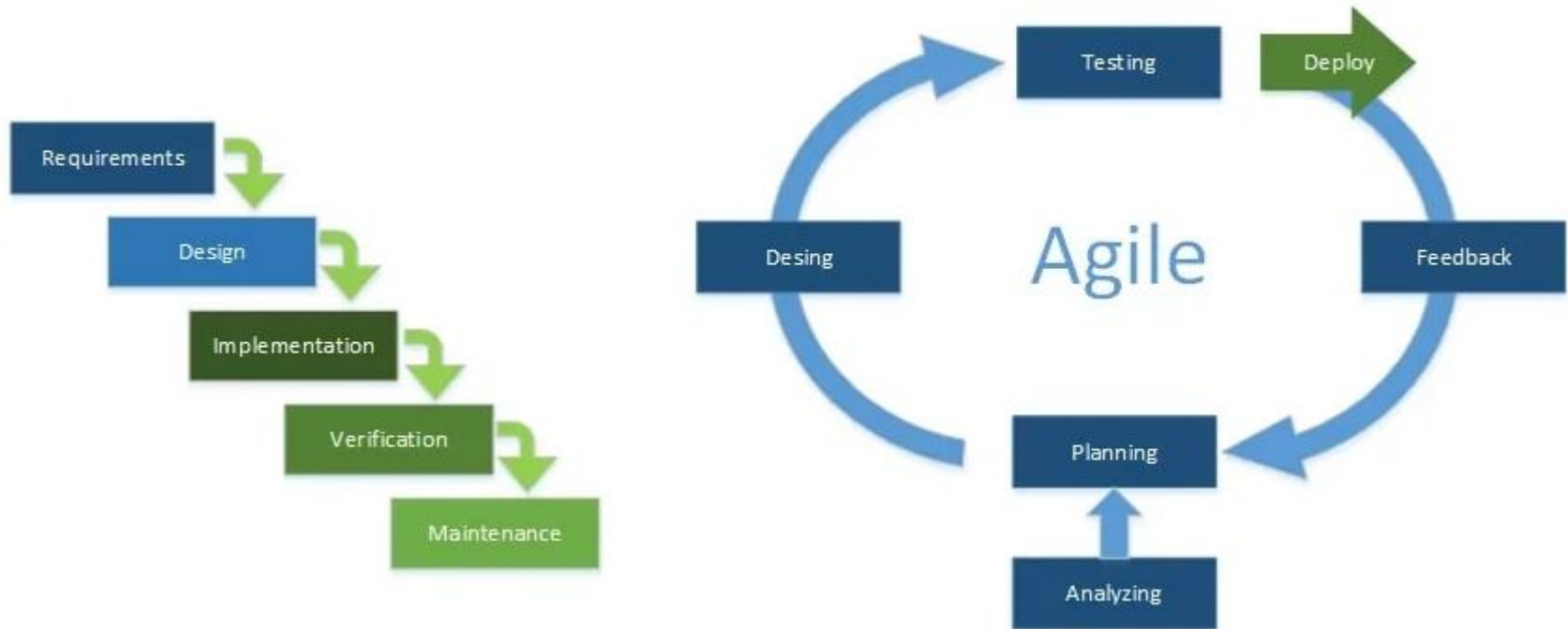
Seguridad desde el principio y todo el tiempo

•
No hace falta un modelo de amenazas para:

- Definir el uso de TLS
- Verificar las entradas de una interfaz
- Definir roles desde el principio
- Ver los flujos alternativos.
- Pensar que debe suceder cuando las cosas no funcionan y hay un error.
- Estar atento a vulnerabilidades y amenazas
- validar supuestos **que tenga el caso y hacer cumplir o forzar las condiciones que se impongan**



Waterfall vs. Agile



Historias de usuario

Normalmente

- Definir Requisitos?
- demostrar la funcionalidad desde la perspectiva de un usuario.
- definir personas
 - Criterios de aceptación
 - Comentarios con elementos que aportan valor
 - Estimaciones de tiempo?
 - Centrarse en el valor??

Dele una chance a la seguridad...

- No termine hasta que analice un modelo de amenazas del caso de uso o Historia
- Incluir criterios de aceptación de seguridad.
- Diseñar para que los datos viajen y se guarden con confidencialidad.
- El operador no pueda negar que realizó la transacción.
- Manejar explícitamente las condiciones de error (estados de falla)
- Testeos unitarios de seguridad



Pensar como el adversario...

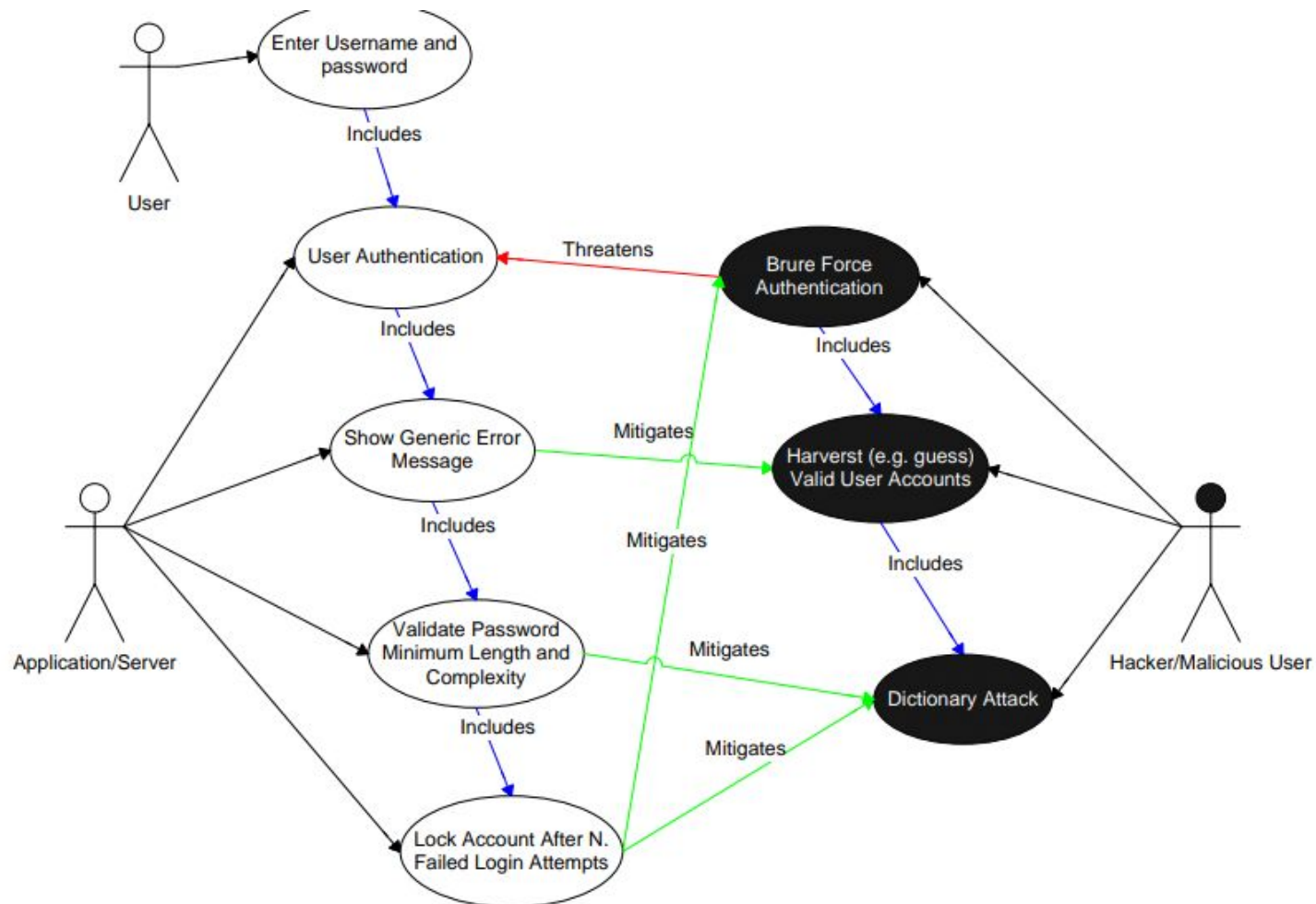
Historias de Abuso

- Una 'historia de abuso' es una historia de usuario desde el punto de vista de un adversario malicioso.
- Intentan ser una base para definir las actividades que deben ser bloqueadas o mitigadas activamente por el software y demostradas mediante pruebas de regresión automatizadas.



Como Cliente Autenticado,

- Veo lo que parece ser mi número de cuenta en la URL, así que lo cambio por otro número para ver qué sucede.
- Trato de entrar como otro usuario o de usar funciones a las que no tengo acceso modificando la url, cambiando parametros, etc.



Story Points: medida abstracta que representa la complejidad y el esfuerzo requerido para completar una tarea.

Que pasa si se detecta una amenaza grave con impacto potencial?

- ¿que pasa si sale mal? (muy mal)
 - → Sorry point... Estimacion del esfuerzo requerido para resolver dicho impacto...
-

Deja de golpear al topo...

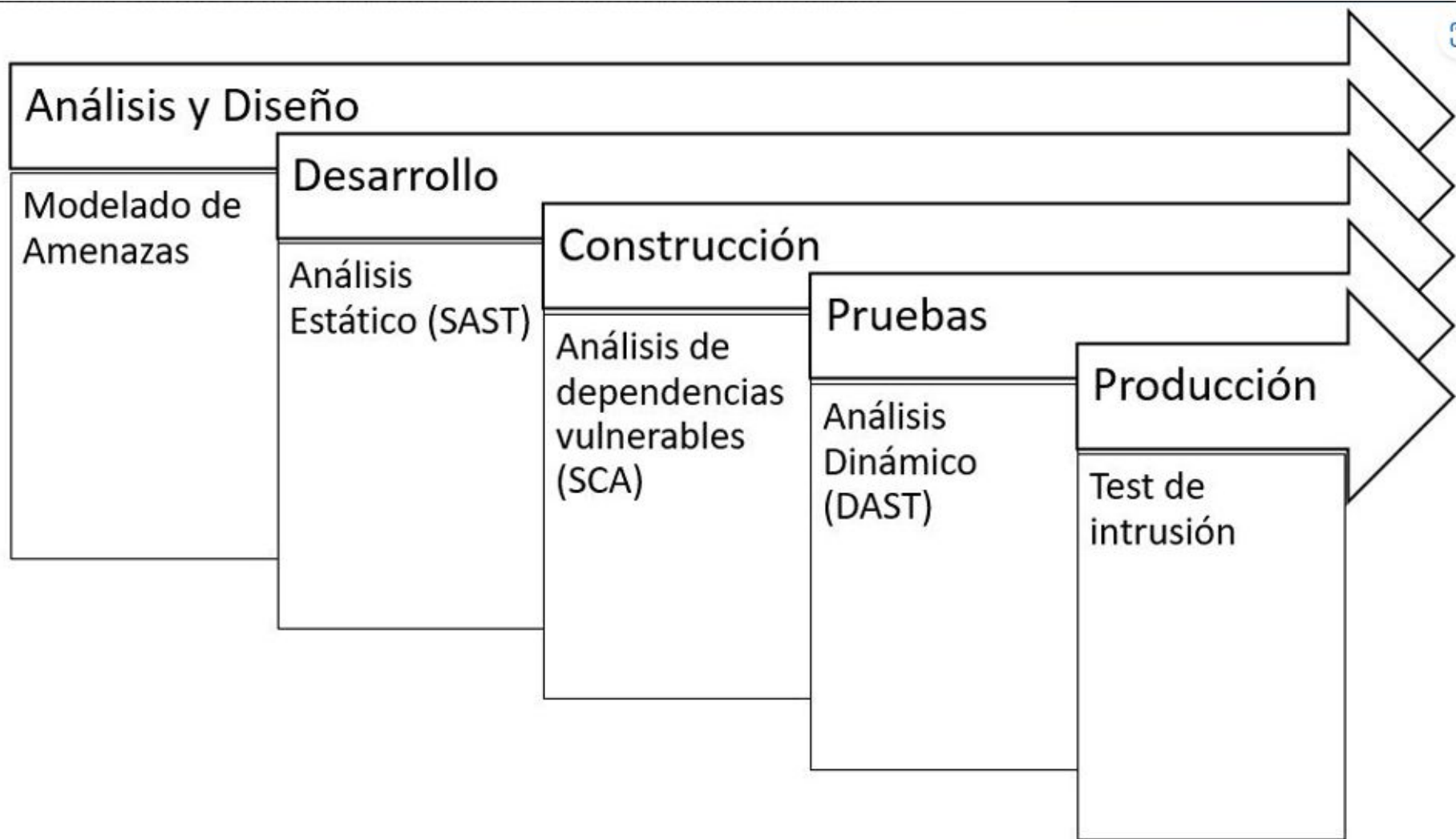
La modelización de amenazas es la clave para una defensa enfocada.

Sin la modelización de amenazas, nunca podrás dejar de jugar al 'whack-a-mole' (Golpea al topo).

Adam Shostack



Seguridad desde el diseño



Anexo:

Top 10 fallas de diseño

Respuesta a crisis de confianza y fraude sistémico



1 La confianza se gana o se otorga, nunca se asume

- Diseñar sistemas seguros implica validar rigurosamente los componentes y no confiar ciegamente en entornos hostiles..
- La cooperación de componentes esencial.
- Riesgo al ejecutar partes en entornos hostiles.
- Advertencia sobre confiar en componentes no verificables.
- Necesidad de precaución al offload de operaciones de seguridad.
- Validación rigurosa de datos no confiables al diseñar sistemas.

2 Mecanismo de Autenticación que no puedan ser eludidos o manipulados

La autenticación valida la identidad. Evitar suplantaciones, usar múltiples factores y almacenar credenciales seguras.

- ✓ Esencial para prevenir el acceso no autorizado.
- ✓ Múltiples factores de autenticación refuerzan la seguridad.
- ✓ Riesgo de suplantación si las credenciales son fáciles de falsificar.
- ✓ Debe incluir mecanismos de seguridad para evitar el acceso no autorizado.
- ✓ Debe tener límites de tiempo y ser segura contra suplantación.

3 Autorizar luego de autenticar

La identidad del usuario es esencial, pero no suficiente; se requiere la autorización adecuada para ciertas acciones.

- ✓ La autorización va más allá de la autenticación.
- ✓ Contexto y privilegios afectan la autorización.
- ✓ La revocación de autorizaciones es esencial.
- ✓ Operaciones sensibles pueden necesitar re-autenticación.
- ✓ La autorización debe ser manejada por una infraestructura común y reutilizable.

4 Separar estrictamente los datos y las instrucciones de control,

y nunca procesar instrucciones de control recibidas de fuentes no confiables.

- ✓ La combinación de datos y control en una entidad puede conducir a vulnerabilidades de inyección.
- ✓ La falta de separación estricta permite que datos no confiables controlen el flujo de ejecución.
- ✓ Puede resultar en vulnerabilidades de corrupción de memoria.
- ✓ Y puede dar lugar a vulnerabilidades de inyección.

5 Validar todos los datos

Es esencial validar explícitamente los datos en el software para evitar vulnerabilidades basadas en suposiciones implícitas sobre los datos..

- ✓ Al diseñar sistemas de software una validación exhaustiva de datos y verificar las suposiciones es crucial.
 - Bibliotecas de validación comunes
 - Validación basada en el tipo de datos
 - Debe ser coherente y considerar la dependencia del estado.

6 Usar bien la criptografía

La criptografía es crucial para la seguridad, pero es compleja. Evitar errores comunes.

- ✓ No diseñar algoritmos criptográficos propios; usar estándares y bibliotecas confiables.
- ✓ Entender y usar las bibliotecas correctamente.
- ✓ Gestionar adecuadamente las claves criptográficas.
- ✓ Utilizar números aleatorios criptográficamente seguros y no reutilizarlos.
- ✓ Prever adaptabilidad y evolución de algoritmos.

7 Identificar datos sensibles y cómo deben ser manejados.

Identificar datos sensibles y cómo protegerlos es crucial.

- ✓ Identificar datos sensibles y sus posibles manipulaciones o exposiciones.
- ✓ Regulaciones, políticas corporativas, contratos y expectativas de usuarios.
- ✓ Necesidades de confidencialidad, disponibilidad e integridad.
- ✓ Usar controles técnicos como mecanismos de acceso, cifrado y respaldos para proteger datos.
- ✓ La protección de datos se aplica no solo en reposo, sino también en tránsito y en entornos interconectados.

8 Tener en cuenta los usuarios

La seguridad de un sistema de software está vinculada a sus usuarios.

- ✓ Las características de usabilidad y experiencia del usuario son esenciales para operar de manera segura.
- ✓ Diseñar sistemas que faciliten la configuración y el uso seguros es un desafío importante.
- ✓ La falta de consideración de los usuarios puede llevar a problemas de seguridad y privacidad.
- ✓ Se deben considerar:
 - Las necesidades de diferentes clases de usuarios
 - Evitar la fatiga del usuario al equilibrar seguridad y usabilidad..

9 Integrar componentes externos cambia la superficie de ataque

La incorporación de componentes externos es algo común. Se debe gestionar su seguridad y efectos adecuadamente.

- ✓ Los sistemas de software a menudo utilizan componentes externos, como bibliotecas, plataformas y aplicaciones.
 - Garantizar que los componentes externos cumplan con los estándares de seguridad actuales.
 - Su inclusión de componentes externos afecta el modelo de amenazas del sistema y su superficie de ataque.
 - Se deben configurar los componentes externos solo para la funcionalidad requerida y validar su origen e integridad.
 - Se debe verificar regularmente si necesitan actualizaciones.

10 Flexibilidad a cambios

diseñar sistemas que puedan adaptarse a cambios en el entorno, actualizaciones seguras y evolución de componentes externos, además de

√ El diseño debe ser

- adaptable a cambios en el entorno y la cantidad de usuarios.
- Actualizaciones seguras, evitando cambios masivos.
- Considerar la verificación de integridad y origen de las actualizaciones.
- Diseñar para cambios en propiedades de seguridad, como contraseñas y claves.
- Prever cambios en propiedades de componentes externos y diseñar con agilidad para adaptarse
- Garantizar la gestión adecuada de secretos y control de acceso..



